# Aretia Finance — Full Security Review

| | |
|---|---|
| **Version** | 1.0, 18 October 2026 |
| **Commit reviewed** | `5e7fb2375926e544e6c7bbebaa119873c61419f4` (`main`, after PR #213) |
| **Prepared by** | Claude Code (AI-assisted review), at the request of the Aretia Finance owner |

> **Read this first.** This is an AI-assisted security review, written in the structure of a professional audit report so that it can be compared with one. It was **not** produced by ChainSecurity or by any audit firm. It does not replace an independent audit, and it gives no guarantee that the code is free of vulnerabilities. The review ran on 18 October 2026 with eight parallel reviewers, each covering one area. Every High and Medium finding was then re-checked against the code, and where a test exists, by running it. The proof-of-concept tests are in `poc/`, next to this file.

---

## Table of contents


1. Executive summary
2. Assessment overview
3. System overview and trust model
4. Findings
    - High
    - Medium
    - Low
    - Informational
5. Previously reported issues: status
6. Properties checked and found sound
7. Recommendations in priority order
8. Appendix: methodology, tooling, reproduction


---

## 1. Executive summary

Aretia Finance is a non-custodial web wallet and trading product. It has three parts:

- **A swap aggregator ("Swings").** It routes through Aretia's own venue adapters, 0x on EVM chains and Jupiter on Solana.
- **A token launchpad.** It mints Solana Token-2022 tokens and creates EVM factory tokens. Launches can be paired with Meteora pools, Uniswap-V2-style pools, bonding curves (EVM, live; Solana, ready to deploy) and Aretia's own single-sided pool (Solana devnet, EVM Sepolia).

- **Market data, account and payments.** Market data and the Launch Box listing, wallet sign-in with favourites and alerts, a referral programme, a card on-ramp, and USDC moves across chains through Circle CCTP.

**The on-chain code is in good shape.** The issues are concentrated in the web layer, where the client acts on data from third-party services.

- **No Critical issue was found.** None of the smart contracts or Solana programs lets a third party drain funds held by the protocol. The bonding-curve and pool maths round against the trader in every place we checked. Solvency is checked after every value move, and the Solana programs check mints by reading their raw bytes, which is stricter than Token-2022 itself. The fixes from the earlier review (`docs/AUDIT_2026-10-18.md`) were re-checked and are correct.
- **The three High findings are all web-client safety gaps.** In each, the page signs what an outside source says without checking enough of it:

  1. the 0x **AllowanceHolder** call (H-1);
  2. **Jupiter**'s instruction list (H-2);
  3. a **restored "Move USDC" record** (H-3).

  Each was reproduced in a test. Each requires a compromised upstream or social engineering, and each can lose the user's whole sell amount or balance.

- **Medium findings** cover:

  - money flows that a privileged key or a token creator can freeze or drain from inside the rules: the fee-recipient freeze (M-2), tokens held outside a curve (M-3) and "burned" Solana pool positions (M-5);
  - griefing that blocks graduation (M-4);
  - the integrity of what the site shows users before they trade (M-6 to M-8);
  - referral abuse (M-1), an alert-system DoS (M-11), a silent re-quote (M-12), and the initialisation of the staking program (M-10).

| Severity | Count | Reproduced by a test |
|---|---|---|
| Critical | 0 | — |
| High | 3 | 3 |
| Medium | 12 | 9 (+1 partly) |
| Low | 31 | 10 |
| Informational | 27 | — |

**What is live today and what is not** decides how urgent each finding is:

| Component | Status at the reviewed commit | Findings that touch it |
|---|---|---|
| Swap (Swings), EVM and Solana | Live | H-1, H-2, M-12, L-1, L-2, L-13, L-14, L-26 |

| Component | Status at the reviewed commit | Findings that touch it |
|---|---|---|
| `AretiaCurve.sol` and `AretiaTokenFactory.sol` (Ethereum, BNB Chain, Polygon) | Live, **immutable** | M-2, M-3, M-4, L-10 to L-12 |
| Solana launch, Meteora pools, Uniswap-V2 pool page | Live | M-4, L-25 |
| Launch Box, markets, token pages, risk ratings, Verified badge | Live | M-6, M-7, M-8, L-15, L-16, L-29 |
| Account sign-in, favourites, alerts, referrals, ramp | Live | M-1, M-11, L-3 to L-7, L-27, L-28 |
| Move USDC (CCTP) | Live behind a canary allow-list | H-3 |
| `aretia-curve` (Solana) | Built and tested; mainnet deploy pending | L-8, L-9 |
| `aretia-pool` (Solana) | Devnet only | M-5, L-21 to L-23 |
| `AretiaPool.sol` | Sepolia only | M-2, M-9, L-19, L-20 |
| `act-staking` | Devnet only | M-10, L-30 |
| `aretia-curve-grad` | Not deployed | L-31 |

---

# 2. Assessment overview

## 2.1 Scope

| Area | Files |
|---|---|
| EVM contracts | `contracts-evm/AretiaCurve.sol`, `AretiaTokenFactory.sol`, `AretiaPool.sol`, their artifacts and standard-input JSON, and `test-mocks/` |
| Solana programs | `program/aretia-curve` (program, `curve-math`, tests), `program/aretia-curve-grad`, `program/aretia-pool` (program, `pool-math`, tests), `program/act-staking`, `sdk/act-staking` |
| Swap engine | `website/src/swings/{engine,router,execution,settlement,chains,providers,dex,plan,intent,orchestrator,economics,crosschain,solana,wallet,shield,tokens,core}`, `website/src/scripts/walletSwap.ts`, `walletTools.ts`, `walletSwings.ts` |
| Transaction builders and inspectors | `website/src/scripts/walletLaunch*`, `walletPool*`, `walletAretiaPool*`, `walletYourPools*`, `walletCurve*`, `walletPromote.ts`, `walletReferrals.ts`, `walletApp.ts` (send), `walletCrossChain.ts`, `walletSend.ts` |
| Market and Launch Box | `website/src/swings/{market,launchbox,indexer,intelligence,charts,referrals,notify,store,db,account}`, `walletLaunchBox*`, `walletMarketTable*`, `walletToken*`, `tokenPageCore.ts`, pages and components |
| Server | every file in `website/api/`, `website/supabase/` (migrations, RLS, cron), `website/vercel.json` |
| Operator tooling | `website/scripts/*.mjs` (deploy and verify helpers, Verified-badge checker), `mainnet/`, `scripts/` |

**Out of scope:**

- third-party programs and contracts, which were treated as trusted with their documented behaviour: Meteora DAMM v2, Uniswap V2 and its forks, 0x Settler, Jupiter, Circle CCTP, MoonPay;
- the wallet extensions;
- the Vercel and Supabase platforms;
- the legacy static pages.

### 2.2 Limitations

- **Meteora was not executed.** Its binary could not be fetched in the review environment, so behaviour that depends on Meteora was checked against its published IDL (`@meteora-ag/cp-amm-sdk` 1.4.9) and the owner's own passing test run with Meteora's mainnet binary.
- **No live chain checks.** Public RPC endpoints were blocked, so every reproduction used local VMs (LiteSVM, `@ethereumjs/vm`) or the real handlers with fake network responses.
- **Not verified on chain:** deployed bytecode and program data (we relied on the repository's own verify scripts and records), and the registry addresses (routers, Permit2, AllowanceHolder, Jito tip accounts), which were checked against public documentation only.

### 2.3 Severity classification

Severity combines likelihood and impact, as in common audit practice:

| Likelihood ↓ / Impact → | High | Medium | Low |
|---|---|---|---|
| **High** | Critical | High | Medium |
| **Medium** | High | Medium | Low |
| **Low** | Medium | Low | Low |

**Impact:**

- **High:** loss or permanent freezing of users' or the protocol's funds.
- **Medium:** temporary freezing, a bounded loss, or misleading information that leads directly to loss.
- **Low:** griefing, small or self-inflicted loss, or defence in depth.

**Informational** items are not vulnerabilities: code quality, documentation drift, gas.

---

## 3. System overview and trust model

### 3.1 Components

**Bonding curves.**

- *EVM:* `AretiaCurve.sol` holds coin per token and sells tokens along a virtual constant-product curve (`vSol × vTok = k`) with a 0.58% fee pushed to the fee address on every

trade. When the curve sells out, anyone may `graduate` it into a Uniswap-V2-style pair with the LP minted to the dead address. After 7 days without graduation, anyone may `abortMigration`, and holders can then sell back.
- *Solana:* `aretia-curve` works the same way. It graduates into a Meteora DAMM v2 pool whose position is permanently locked and held by a migrator PDA. `claim_pool_fees` pays that position's fees, which are wSOL only, to Aretia's fee wallet.
- *Variant:* `aretia-curve-grad` is identical but graduates into the Aretia Pool.

**Aretia Pool.** Each token has one pool. It is constant product with a virtual SOL (or coin) reserve `l`, so a pool can open with tokens only. Ranges are optional. Liquidity is held in positions that can be locked (never unlocked), transferred, and can name a fee recipient. A launch-fee schedule can start high and decay. Versions exist on Solana (`aretia-pool`, with wSOL vaults) and EVM (`AretiaPool.sol`).

**Token launch.**

- *Solana:* a two-transaction Token-2022 launch with metadata and the 0.05 SOL fee. Mint, freeze and (optionally) update authority are revoked.
- *EVM:* `AretiaTokenFactory` deploys a fixed-supply plain ERC-20 for a per-chain fee.

**Swings.** The router quotes Aretia's own venue adapters (direct EVM and Solana pools, Curve, Uniswap V2/V3/V4), 0x, and Jupiter. It applies a 0.58% fee on native, wrapped-native and stablecoin legs, then builds, simulates and signs the transaction through the user's wallet.

- Solana transactions are compared byte for byte after the wallet signs.
- EVM swaps can be batched (EIP-5792) or sent step by step.
- **Shield** scans approvals and offers revocation; it is also the address book behind the look-alike warnings.
- **SafeSend** checks recipients.

**Cross-chain.** "Move USDC" uses Circle CCTP V2 (approve, then burn, attest and mint). A move can be saved on the server as a record and restored later with its recovery code.

**Server** (Vercel functions with Supabase, service role only, RLS on with no policies):

- an RPC proxy;
- quote, submit and 0x proxies;
- wallet sign-in with HMAC session tokens;
- favourites, alerts (Web Push and Telegram) and referrals (on-chain fee verification);
- the Launch Box indexer (cron);
- market, candle and pool data;
- the MoonPay URL signer.

### 3.2 Roles and trust assumptions

| Actor | Trusted for | Can harm if malicious or compromised |
|---|---|---|
| Fee recipient keys (EVM `0xeb8725…0945`, Solana `2tcBrd1J…oZX2`) | Receiving fees | **EVM:** freeze every trade on every curve and pool (M-2). **Solana pool:** halt swaps by closing the wSOL account (L-21). |
| Program upgrade authority (Solana) | Must be the Squads 2-of-3 vault `GtKGE6m…` | Replace any program and take every vault |
| Token creators and curve openers | Nothing | Hold supply outside the curve (M-3), reclaim "burned" positions (M-5), rug unlocked pools (M-9), squat pools (known) |
| 0x, Jupiter, DexScreener, GeckoTerminal | Quotes and market data | H-1, H-2, M-8, L-16, L-29 |
| Public and keyed RPC nodes | Chain state and simulation | A lying node can fake simulation results and risk facts (documented in `docs/audit/known-risks.md`) |
| Aretia's server and Supabase | Sessions, records, alerts, referrals | Can serve forged records (H-3 shows that anyone, not only the server, can write one) |
| The site's own JavaScript origin | Everything the inspectors check | Any script injection defeats every client-side check (L-17) |

---

## 4. Findings

Each finding gives its location as `path:line` at the reviewed commit, and says whether a test reproduces it (see `poc/README.md`).

### 4.1 High

> **H-1 — 0x AllowanceHolder `exec`: operator, target, recipient and minimum are not checked; a bad 0x answer can take the whole sell amount**

**Component:** Swings, EVM. **Live:** yes, whenever `ZEROX_API_KEY` is set (aggregators default on). **Likelihood:** low to medium. **Impact:** high. **Reproduced:** yes (`poc/swap/zerox-exec.test.ts`).

**Location:** `website/src/swings/providers/evm0x.ts:77-84` (`decodeAllowanceHolderExec`), `:184-229`; `website/src/swings/router/router.ts:193-194`.

**Description:** AllowanceHolder's `exec(operator, token, amount, target, data)` grants `operator` a transient allowance of `amount` of `token` from `msg.sender`, then calls `target` with `data`. The decoder checks the selector and words 1–2 (token and amount) only. The following are never checked:

- word 0 (`operator`);
- word 3 (`target`);

- the Settler calldata, which carries the real `recipient`, `buyToken` and `minAmountOut`.

The router's slippage check compares the user's tolerance with the `minBuyAmount` **JSON field**, not with the calldata. The `eth_call` simulation succeeds because an attacker contract simply keeps the tokens.

**Exploit:** a malicious or compromised 0x response (or a compromised `/api/swings-0x` deployment) returns `exec(attacker, USDC, 1000e6, attackerContract, …)` with realistic JSON amounts. The user approves AllowanceHolder for the exact 1,000 USDC, pays the Aretia fee and signs. The attacker's operator pulls the 1,000 USDC. For a native sale, `msg.value` goes to the attacker's target.

**Recommendation:**

1. Require `operator == target`, and `target` equal to the current 0x Settler for the taker-submitted feature on that chain (read from the Settler deployer registry, or an allow-list).
2. Decode the Settler `execute(AllowedSlippage{recipient, buyToken, minAmountOut}, actions, …)` call. Require `recipient == taker`, `buyToken == requested token` and `minAmountOut ≥ expectedOut × (1 − slippage)`.
3. Make the simulation assert the taker's buy-token balance change, as the direct EVM builder already does.
4. Correct `docs/audit/trust-model.md`, which says aggregator output is "bounded by the user's minimum".

This generalises the token and amount check added in commit `04eb145`.

> **H-2 — Jupiter instructions go into the signed transaction without an instruction allow-list; changes that move no balance pass undetected**

**Component:** Swings, Solana. **Live:** yes, by default, whenever Aretia's fee does not apply, which is the common "sell a memecoin" path. **Likelihood:** low. **Impact:** critical (whole-wallet takeover). **Reproduced:** yes (`poc/swap/jupiter-inject.test.ts`).

**Location:** `website/src/scripts/walletSwap.ts:271-302` (every `computeBudget`, `setup`, `swap`, `cleanup` and `other` instruction is copied); the judgement relies only on balance changes (`walletTools.ts:316-364`), with the watch list capped at `MAX_WATCHED` (`walletSwap.ts:43, 304-306`).

**Description:** the only guard is "the simulated balance changes look like the swap". The following instructions change control without changing any watched balance, so they pass:

- SPL `Approve` or `ApproveChecked`, giving a delegate u64::MAX;
- `SetAuthority`, changing a token account's owner or close authority;
- `SystemProgram.Assign` of the wallet to a hostile program, which needs only the user's existing signature and hands over the wallet's lamports later;
- transfers from token accounts that are not associated accounts, or beyond the 48 watched;
- changes to stake or nonce authority.

The `numRequiredSignatures === 1` check does not help, because the user is the signer.

**Exploit:** a compromised Jupiter API (`lite-api.jup.ag` is called directly from the browser) adds `Approve(victim's BONK ATA → attacker, u64::MAX)` and `Assign(victim → attackerProgram)` to a genuine route. The simulation shows exactly the expected SOL/USDC change, and `planSwap` returns no blockers. After the user signs, the attacker drains the BONK and the wallet.

**Recommendation:**

1. Allow-list instructions by program and by instruction tag:
    - ComputeBudget;
    - System `transfer` and `createAccount` to targets other than the wallet;
    - ATA `createIdempotent`;
    - SPL Token and Token-2022 `syncNative`, and `closeAccount` of the user's wSOL ATA back to the user;
    - the Jupiter v6 program for the swap instruction.
2. Reject `Approve*`, `SetAuthority`, `Assign`, nonce instructions and unknown programs.
3. Check the user-owned accounts that the swap instruction writes to.
4. Watch every writable token account in the message, not only the user's associated accounts.

> **H-3 — Move USDC: a restored recovery record is trusted as a quote; a crafted code makes the page approve or burn to the attacker**

**Component:** cross-chain (CCTP). **Live:** yes, behind a canary allow-list, and only where server records are enabled. **Likelihood:** medium (social engineering: "paste this recovery code to rescue your stuck transfer"). **Impact:** high (the victim's USDC balance on the source chain). **Reproduced:** yes, offline (`poc/tx-builders/cctp.poc.test.ts`).

**Location:**

- `website/api/_swingsRecords.ts:46-55, 100-125`: anyone can PUT a record with any id they choose and any content; `cleanRecord` checks shape only.
- `website/src/swings/orchestrator/remote.ts:84-93` (`restoreFromCode`).
- `website/src/scripts/walletCrossChain.ts:186-189, 234-245`: Restore, then Continue, calls `orchestrator.start` directly. `assessSettlement` (line 292) and the canary check (line 290) run only when quoting, not on a restored record.
- `website/src/swings/orchestrator/orchestrator.ts:96-124`: the expiry comes from the record itself, and the only identity check is `wallet == intent.sender`.
- `website/src/swings/settlement/cctp.ts:282-330`: the approve's spender is `raw.tokenMessenger` and its target is `raw.usdc`, the burn goes to `raw.tokenMessenger`, and `mintRecipient` is `intent.recipient`. None of these is derived from `CCTP_CONTRACTS` or `CCTP_USDC`, and `MOVE_CAP_RAW` is not applied again.
- `website/src/swings/wallet/safety.ts:48-53`: for EVM it checks only chain id, `from` and that `to` is on the allow-list. It does not decode the spender or the `depositForBurn` arguments.

**Exploit:**

- **Variant A.** The attacker stores the record `{state:'QUOTED', quote:{providerId:'circle-cctp-standard', expiresAt: far future, intent:{sender: victim, recipient: attacker}, raw:{usdc: real USDC, tokenMessenger: ATTACKER, amount: victim balance, needsApproval: true}}}` and sends the victim its code. The victim restores it and presses Continue. Step 1, `USDC.approve(ATTACKER, balance)`, goes to the genuine USDC contract, so the gateway passes it, and the attacker calls `transferFrom`.
- **Variant B.** The record keeps the genuine messenger and changes only `intent.recipient`. The burn goes to Circle's real TokenMessenger, which looks legitimate in the wallet, and mints to the attacker on the destination chain. This also works from Solana.

**Recommendation:**

1. Never sign from a restored `QUOTED` or `AWAITING_SIGNATURE` record. Re-quote instead, and run `assessSettlement` and the canary check again.
2. In `buildSettlement`, derive USDC, TokenMessenger and domains from constants for the intent's chains, apply the cap again, and bound `maxFee`.
3. Require `intent.sender` and `intent.recipient` to be the user's connected accounts on both chains, or show the full recipient for explicit confirmation.
4. Add a CCTP inspector that decodes the approve spender and amount and every `depositForBurn` argument.
5. Authenticate stored records: bind them to a wallet signature, or to a server-issued HMAC.

---

## 4.2 Medium

> **M-1 — Referral credit is counted up to five times by changing the letter case of an EVM transaction hash**

**Component:** referrals API. **Likelihood:** high. **Impact:** medium (a self-referring pair gets 100% of its swap fee back; it would be higher with automated payouts). **Reproduced:** yes (`poc/api/poc.poc.test.ts`, "F1": 12 case variants gave 5 credited and 7 flagged, and the owed total equalled the whole fee).

**Location:** `website/api/_referralVerify.ts:252` (accepts mixed-case hex); `website/api/_swingsReferrals.ts:104` (`new Set` on the raw strings) and `:150-151`; `supabase/migrations/0011_referrals.sql:33` (primary key `(chain, tx, asset)`, case-sensitive); burst rule in `src/swings/referrals/core.ts:96-97`; export in `scripts/referralExportCore.mjs:35-51`.

**Description:** EVM nodes read a transaction hash in any letter case, but the database treats every spelling as a new row. The burst rule only flags the sixth identical credit, so five copies are credited at 20% each.

**Recommendation:**

- Lower-case EVM hashes before removing duplicates, storing and querying, and reject anything else.
- Add `check (tx = lower(tx))` for EVM chains and merge existing duplicates by `lower(tx)`.
- Make the export group by `lower(tx)`.

### M-2 — The fee recipient is paid inside every trade; under EIP-7702 its key holder can freeze (or selectively censor) every buy and sell, permanently on the immutable contracts

**Components:** `AretiaCurve.sol` (deployed, immutable) and `AretiaPool.sol` (Sepolia). **Likelihood:** low (it needs a malicious or compromised fee key, or a wallet "smart account" upgrade by mistake). **Impact:** high. **Reproduced:** yes (`poc/evm-curve/poc4.mjs`; `poc/evm-pool/poc.mjs`, PoC2).

**Location:** `AretiaCurve.sol:208, 233, 342-345` (`_send(feeRecipient, fee)` inside `buy` and `sell`; failure reverts). `AretiaPool.sol:163, 192, 215, 411-414`.

**Description:** the contracts assume an EOA can never refuse coin (`BONDING_CURVE_EVM.md` attack #9; `pool-evm.mjs` checks once that the address has no code). EIP-7702 is live on Ethereum (Pectra), BNB Chain (Pascal) and Polygon PoS (Bhilai), so one signed authorisation turns the fee EOA into code that reverts. Then:

- every `buy` and `sell` on every curve on that chain reverts with "send";
- this includes sells after `abortMigration`, the holders' only exit;
- the code can also revert selectively (for example by `tx.origin`), which hands the key holder a censorship and honeypot switch;
- in the pool, `removeLiquidity` and `claimFees` still work, but holders cannot sell.

This contradicts the "no privileged role" statement in `AretiaCurve.sol:12-13`.

**Recommendation:**

- **New versions:** pull payments. Accrue `protocolOwed` (counted in `totalOwed`) and let anyone call `withdrawProtocolFees()`. Or make a failed or gas-limited fee send credit `protocolOwed` instead of reverting.
- **Deployed curves:**
  - document the fee key as a privileged freeze key;
  - keep it cold and never sign a 7702 authorisation with it;
  - monitor `eth_getCode(feeRecipient)` on all three chains and alert on any change.

### M-3 — Tokens held outside a curve can be sold into it, draining buyers' coin and leaving them unable to sell

**Components:** `AretiaCurve.sol` (deployed); the same mechanism exists in the Solana curves. **Likelihood:** high (permissionless launchpad). **Impact:** high for the buyers affected. **Reproduced:** yes, on EVM (`poc/evm-curve/poc3.mjs`).

**Location:** `AretiaCurve.sol:143-152` (any `total` from 1 to 2^96, with no link to `totalSupply`), `:240-241` (sells capped only by `curveTokens`); `AretiaTokenFactory.sol:86-92` (supply has no upper bound, so it can exceed `MAX_TOTAL`, see I-4). Solana: `curve-math/src/lib.rs:151-158` with `create_curve` accepting a `total_tokens` below the supply.

**Description:** the curve lets anyone sell tokens back until `realTok == curveTokens`. Two consequences:

- A holder of supply outside the curve (the creator, or anyone given tokens) can sell first and take buyers' `realSol`.
- Once the curve is "full", every real buyer's `sell` reverts with "more than sold".

The design document mentions creator dumps but not that buyers are then locked out.

PoC: supply 1e9, with 1e8 on the curve and a 5 ETH target. A buyer pays 3 ETH and receives 67.4M tokens. The creator then sells 67.4M tokens from outside the curve and receives 2.965 ETH. `realSol` is left at 1 wei, and the buyer's sell reverts.

**Recommendation:**

- New versions: require `total == totalSupply()` held entirely by the caller, and cap factory supply at `MAX_TOTAL`.
- Now, on the web: show on every trade card "X% of supply is outside this curve, and holders of it can sell ahead of you", with the opener's live balance. Warn strongly when the share is above 0. This is `walletCurveEvm.ts:260-272`, and the same applies to the Solana card.

> **M-4 — Graduation, and the Uniswap-V2 pool page, can be blocked for any token with 1 wei of WETH, without holding the token**

**Components:** `AretiaCurve.sol` `graduate` (deployed); `walletPoolEvmFlow.ts` (live). **Likelihood:** high. **Impact:** medium (holders frozen for 7 days and the launch never graduates; the V2 page refuses for good). **Reproduced:** yes, for the curve, including the recovery (`poc/evm-curve/poc1.mjs`, `poc2.mjs`, `Rescue.sol`).

**Location:** `AretiaCurve.sol:268-290, 348-358`; `website/src/scripts/walletPoolEvmFlow.ts:106-111, 178-190`; `walletCurveEvmFlow.ts:418, 551-565`; `walletCurveEvmCore.ts:397`.

**Description:** this extends earlier finding #4, which assumed 1 wei of each side. **The token side is not needed:**

1. `createPair(token, WETH)`;
2. send 1 wei of WETH to the pair;
3. call `sync()`.

The reserves are then (0, 1). `UniswapV2Library.quote` requires both reserves to be above 0, so `addLiquidityETH` reverts with `INSUFFICIENT_LIQUIDITY`. This works from the moment the token exists. Holding back tokens does not prevent it. The page does not recognise the revert text and only offers the 7-day unwind. The V2 page refuses with "already exists with liquidity".

**Recovery is possible without changing the contract (reproduced):** one transaction can donate the missing side so that the pair's reserves match `realSol : lpTokens`, call `sync()`, then call `graduate()`. In all three blocking cases it graduated (status `Migrated`) for 208k–263k gas, and because it is atomic it cannot be blocked again in between. An attacker's real-value skew can be taken out by the rescuer, so attacking costs the attacker money.

**Recommendation:**

1. Deploy a permissionless "rescue-graduate" helper of this shape (also `skim` an unsynced pair). Have the page detect a pair with reserves (`getPair`, `getReserves`) and offer it.
2. For the V2 pool page, when `totalSupply == 0`, skip the router: transfer both sides to the pair and call `mint(DEAD)` from a small helper, then check the reserves.
3. Map `INSUFFICIENT_LIQUIDITY` and tighten the `/exists/i` revert regex.
4. In a new curve version, let `abortMigration` run immediately when the pair has reserves before graduation, or graduate through `pair.mint` directly.

### M-5 — Solana Aretia Pool: a position "burned" to the zero address can be reclaimed, unlocked liquidity included

**Component:** `aretia-pool` (devnet). **Likelihood:** medium (sending to `1111…1111` is a common "burn" convention that dashboards accept). **Impact:** high for holders. **Reproduced:** yes (`poc/solana-pool/poc.rs`, `poc_fake_burn_then_reclaim_and_remove`: after locking half, a 100 SOL buy and the fake burn, the creator withdrew 49.875 SOL of buyers' SOL).

**Location:** `program/aretia-pool/programs/aretia-pool/src/lib.rs:322-331` (`transfer_position` accepts `Pubkey::default()`); `:214-226` (`add_liquidity` treats `pos.owner == Pubkey::default()` as "new position", rewrites `owner`, `checkpoint` and `recipient`, and keeps `shares` and `locked`).

**Description:** a "new position" is detected by a sentinel value rather than by the account having just been created. A position sent to the default key looks new to `add_liquidity`, and whoever passes the id rule can take it over. For the usual case, where the id is the creator's own address, that is the creator. Locked shares stay locked, but everything unlocked can then be removed. A side effect is that the `checkpoint` rewrite discards fees settled to the old owner but not yet claimed.

**Recommendation:**

- Initialise only on a real fresh account (for example `pos.pool == Pubkey::default()`, or an explicit `initialized` flag).
- Make `transfer_position` reject `Pubkey::default()`, or turn it into an explicit irreversible burn that locks all shares.
- Document `lock_liquidity(all)` as the only true burn, and correct `docs/ARETIA_POOL.md:157`.

### M-6 — Solana tokens with dangerous Token-2022 extensions or retained authorities are listed as Aretia launches and shown as safe

**Components:** Launch Box indexer, token page, risk engine (all live). **Likelihood:** medium to high (one launch fee, or none for the page and rating). **Impact:** medium (it misleads buyers into honeypots on Aretia's own domain). **Reproduced:** yes (`poc/api/poc.poc.test.ts` "F-launchbox"; `poc/market/market.poc.test.ts`; `poc/market/risk2.poc.test.ts`).

**Location:**

- `website/src/swings/launchbox/core.ts:207-234` (`acceptSolanaLaunch`) and `indexer.ts:244-251`. A launch is accepted on the fee transfer plus one Token-2022 `InitializeMint`, with no check of mint authority, freeze authority or extensions.
- `website/src/scripts/tokenPageCore.ts:86-101`: only extension 1 is flagged, and "Supply fixed / freeze given up" are shown in green.
- `website/src/swings/tokens/enrich.ts:34`: `hasExtensions = data.length > 82`.
- `website/src/swings/tokens/risk.ts:137`: any extension is a 5-point warning. Every Aretia launch carries the metadata extensions, so this signal is always on.

**Description:** none of these views notices the extensions that matter:

- a permanent delegate (12), which can take any holder's tokens;
- a transfer hook (14), which can block sells;
- Pausable (26);
- default-frozen accounts (6);
- non-transferable (9);
- scaled UI amounts (25).

A hand-built launch that keeps the mint and freeze authority is listed as "created through Aretia Launch".

**Recommendation:**

- In the indexer, apply the same raw-mint rule the on-chain programs use: no mint or freeze authority, and only extensions 18 and 19. Otherwise hide or flag the token.
- Parse the extension types in the token page and the risk engine. Mark 12, 14, 26, 6 and 9 as "bad" and 24 and 25 as warnings, and treat 18 and 19 as neutral.

### M-7 — The "Verified by Aretia" checker misnumbers Token-2022 extensions and passes EVM proxies

**Component:** `website/scripts/verifyTokenCore.mjs` (operator tool behind a trust badge). **Likelihood:** medium. **Impact:** medium. **Reproduced:** yes (`poc/market/vt.mjs`: Pausable, ScaledUi, MintClose, ConfidentialFee, ConfidentialMintBurn and an EIP-1167 proxy all pass).

**Location:** `verifyTokenCore.mjs:8-17` (wrong numbers: 4 is ConfidentialTransferMint, 11 is CpiGuard; missing 3, 16, 24, 25 and 26), `:106-124` (no proxy detection), `:127-146` (the report never lists the extensions, though `docs/verified-badge.md` relies on the reviewer seeing them).

**Recommendation:**

- Use an allow-list (18, 19, the group extensions 20–23 and interest-bearing 10 as flag-only) and fail on anything else.
- Print the extension list in the report.
- Add the EIP-1167 and EIP-1967 checks from `enrich.ts`.
- Read the supply at two different blocks.

### M-8 — A pool paired with a made-up token takes over the price, liquidity and market cap shown in Launch Box, the token page and the chart

**Component:** market data (live). **Likelihood:** medium. **Impact:** medium (it drives trading decisions and feeds the risk rating). **Reproduced:** yes (`poc/market/market.poc.test.ts`: both `bestPools` and `chooseDexPair` pick the fake pool).

**Location:** `website/src/swings/launchbox/markets.ts:39-66` (`bestPools` ranks by `liquidity.usd` only); `website/src/swings/charts/dexscreener.ts:50-63` (the "reasonable" filter is 5% of the deepest pool, so a fake pool reporting more than 20× the real one removes every real pool from the set); `website/src/scripts/walletApp.ts:228-240`.

**Recommendation:**

- Rank first by whether the pool is paired with a major quote token (stablecoins and wrapped native coins), then by depth.
- Compute the threshold over major-paired pools only.
- In Launch Box, show liquidity and market cap from major-paired pools, or label the quote token.

### M-9 — EVM Aretia Pool: pools opened unlocked by a direct call can be emptied, and the trade card does not show how much is locked

**Component:** `AretiaPool.sol` and its web card (Sepolia only). **Likelihood:** medium. **Impact:** high for buyers. **Reproduced:** yes (`poc/evm-pool/poc.mjs`, PoC3: the creator took 9.9 ETH of a buyer's 10 ETH, and `quoteSell` of the buyer's tokens returned (0, 0)).

**Location:** `AretiaPool.sol:159` (`lock` is optional), `:60-75` (no per-pool locked total), `:259-280`; `website/src/scripts/walletAretiaPoolEvm.ts:286-345` (only the viewer's own lock is shown).

**Recommendation:**

- Enforce `lock` (or a minimum locked fraction of `l0`) in `createPool`, or track `pool.locked` and show "x% of liquidity locked" with a warning.
- Reword "the price never falls below the start": the floor price holds, but the coin behind it can be withdrawn.
- The Solana pool has the same property (unlocked liquidity can be removed). Its trade card should show the locked share too.

### M-10 — `act-staking` initialisation can be front-run or blocked for good

**Component:** `program/act-staking` (devnet; matters for a fresh mainnet deploy). **Likelihood:** medium at deploy time. **Impact:** high (wrong authority or mint for good, or initialisation impossible; recovery needs an upgrade or a new program id). **Reproduced:** no (it follows directly from the account constraints).

**Location:** `program/act-staking/programs/act-staking/src/lib.rs:54-85, 324-357` (no constraint on `authority` or `mint`); `:345-352` (the vault uses `init`, so a pre-opened associated account blocks it, the same class as earlier finding #1). `STAKING_DESIGN.md:73` calls it "authority-gated". `Anchor.toml` gives mainnet the same id as devnet.

**Recommendation:**

- Constrain `authority` (a hard-coded admin, or equal to the program's upgrade authority) and `mint == ACT_MINT`.
- Open the vault with `init_if_needed` or idempotently.
- Deploy and initialise in one sitting, then verify.

### M-11 — Anyone can stop alerts for every user, and make the alert job send requests to any https host

**Component:** alerts API (live). **Likelihood:** high (sign-ins are free with throwaway keys). **Impact:** medium (alerts stop for everyone; blind outbound POSTs). **Reproduced:** partly (`poc/api/poc.poc.test.ts` "F-push": any https endpoint is accepted, including `169.254.169.254.nip.io`).

**Location:** `website/api/_swingsNotify.ts:293` (`notify_channels … limit=3000`, no `order`), `:322` (a single shared 400-token price list), `:198-207`, `:105`, `:236` (`op: test`); `website/src/swings/notify/vapid.ts:55, 61-69` (no host check; `fetch` follows redirects); favourites up to 500 per owner (`_swingsAccount.ts:146-160`).

**Description:** about 500 throwaway owners × 6 channels fill the 3,000-row read, and with no `order`, real owners may never be read. Alternatively, a few owners with 400+ random favourites fill the 400-token price lookup. Each `op: test` makes Aretia's server POST to any registered host, and the response code decides whether the channel is deleted or marked healthy.

**Recommendation:**

- Accept push endpoints only on the push-service hosts (`fcm.googleapis.com`, `*.push.services.mozilla.com`, `*.notify.windows.com`, `web.push.apple.com`) and use `redirect: 'manual'`.
- Read channels in ordered pages.
- Share the token budget fairly across owners, and cap watched favourites per owner.

### M-12 — Pressing Swap can sign a newly fetched quote the user never saw;

> **the minimum moves with it**

**Component:** Swings UI (live). **Likelihood:** medium (any delay over 8 s). **Impact:** low to medium (the user gets materially less than shown; the provider can also change silently). **Reproduced:** no (code reading).

**Location:** `website/src/scripts/walletSwings.ts:745-753` (`swapNow` calls `getQuote(true)` and `prepareQuote`, replacing `s.quote`, then `confirm()`), `:668-674`, `:719-725`. `AretiaRouter.degradationBps` exists but only guides choosing an alternative route (`:1252`).

**Description:** the minimum is recomputed from the new quote, so the slippage limit no longer protects the price the user approved. For Jupiter and 0x quotes (`priceImpactBps: null`) and thin pools, nothing bounds the difference.

**Recommendation:** when a refresh happens inside `swapNow`, compare it with the displayed quote (output, minimum, provider). Above a small tolerance, require a fresh confirmation, or keep the displayed minimum as a hard floor.

---

## 4.3 Low

| ID | Title | Location | Reproduced | Recommendation |
|---|---|---|---|---|
| L-1 | EVM transactions are not bound to a `chainId`; the network is checked before, not with, each send. A switch mid-flow sends value or a swap on another chain. | `swings/chains/evmWallet.ts:31-38, 188-191`; `chains/evm.ts:96-175`; `walletLaunchEvmFlow.ts:158-175`; `walletCurveEvmTx.ts:103-110`; `walletEvmSend.ts:149-156`; `walletAretiaPoolEvmTx.ts:137-146` | No | Put `chainId` in every `eth_sendTransaction`, check it again before every send, and abort on `chainChanged` or `accountsChanged`. |
| L-2 | The EVM swap fee is sent as its own transaction before the swap. If the user declines, or the swap reverts later, the fee is lost; a retry gets a new `quoteId` and charges again. | `swings/chains/evm.ts:158-175`; quote TTL 15 s (`evm0x.ts:20`, `directEvm.ts:30`) | No | Simulate again after the fee receipt. Take the fee inside the swap where the venue allows it. Otherwise key "fee paid" by taker, chain, token, amount and a time window (kept in `sessionStorage`), and warn that it is not refundable. |

| ID | Title | Location | Reproduced | Recommendation |
|---|---|---|---|---|
| L-3 | Sign-in: replay protection is per instance and keyed on the signature's spelling (dropping `0x`, upper-casing or using high-s replays it); the domain comes from the `Origin` header; the message is not in a standard format, so wallets do not check the domain. A phished signature gives a 7-day session. | `api/_swingsAccount.ts:110-129`; `api/_swingsAuth.ts:73-89`; `swings/account/loginMessage.ts:20-31` | Yes (`api` "F-replay") | Use EIP-4361 and Sign-In with Solana, server-issued single-use nonces in the database, a domain from configuration, and revocable sessions. Also set `zip215: false` and reject small-order ed25519 keys (I-21). |
| L-4 | The ramp endpoint signs MoonPay widget URLs for any wallet without a session, so phishing pages get Aretia-signed purchase links to the attacker's address. | `api/_ramp.ts:180-217, 289-297, 321-329` | Yes (`api` "F-ramp") | Require a session and sign only for the signed-in address. Log what is issued. |
| L-5 | Referral caps and the burst rule read an unordered 2,000-row page, so they may miss earlier credits. Capped and flagged attempts still insert rows, without limit. | `api/_swingsReferrals.ts:147` | No | Use server-side counts or an ordered query, stop inserting past the daily limit, and rate-limit per owner. |
| L-6 | Referral fee payments are verified at "confirmed" with no confirmation depth, against a single public node, and are never re-checked before payout. | `api/_referralVerify.ts:188-197, 256-261, 294-296` | No | Use `finalized` on Solana and a per-chain block depth on EVM, keyed providers, and a re-check in `referral-export.mjs`. |
| L-7 | A trader can bind their own second wallet as referrer on the first `record` call, including for a transaction made up to 6 hours earlier, and gets 20% back for 90 days. Combined with M-1 this becomes 100%. | `api/_swingsReferrals.ts:112-125, 141` | No | Bind only from a link visit recorded before the swap (signed by the server), and never credit transactions made before the binding. |
| L-8 | Solana curve: a sell that pays the seller 0 lamports is accepted, and the tokens are lost (it contradicts BONDING_CURVE.md §8 row 12). | `curve-math/src/lib.rs:162-169` (both curves); `lib.rs:186-187` | Yes (`solana-curve/poc.rs`: 262,144 units sold for 0) | Return `None` when `gross - fee == 0`. |
| L-9 | Solana curve web: anyone can permanently break the "Send pool fees to Aretia" screen by sending 1 unit of any Token-2022 token to the migrator's associated account. The NFT lookup then finds two candidates, and the pool check at `walletCurveFlow.ts:402` can never fail. | `walletCurve.ts:136-140`; `walletCurveFlow.ts:400-402` | No | Find the position from Meteora (`getProgramAccounts` filtered by `pool`, then check that the NFT is held by the migrator), or read it from the `Migrated` event. Check the position's pool field. |

| ID | Title | Location | Reproduced | Recommendation |
|---|---|---|---|---|
| L-10 | EVM curve liveness: graduation costs about 3M gas with no reward, and after 7 days anyone can abort even a curve that would graduate, permanently stranding its `lpTokens`. | `AretiaCurve.sol: 192, 222, 268, 293-299` | No | New versions: a caller bounty, abort only when graduation is impossible, and retry after an abort. |
| L-11 | EVM curve precision at tiny `total` or zero decimals: with total=5 the pool opens 33% above the curve's last price and the raise falls 29% short. | `AretiaCurve.sol: 149-154` | Yes (`evm-curve/poc3.mjs`) | Enforce a minimum `total` (≥ 1e6 base units). |
| L-12 | EVM curve web: the page offers only the 7-day unwind after a failed graduation estimate, and does not map `INSUFFICIENT_LIQUIDITY`. | `walletCurveEvmFlow.ts:418, 551-565`; `walletCurveEvmCore.ts:397` | — | See M-4 (rescue helper). |
| L-13 | If `/api/swings-status` cannot be reached, aggregators default to on, enabling the H-2 path even when the operator turned it off. | `swings/runtime.ts:32, 36, 43` | No | Fail closed (`aggregators: b.aggregators === true`). |
| L-14 | The quote API (and records and tokens) take the client IP from the first `X-Forwarded-For` entry, which the caller sets, so the rate limit can be bypassed and keyed RPC simulations abused. | `api/swings-quote.ts:20-24`; `api/swings-records.ts:21, 27`; `api/swings-tokens.ts:22, 29` | — | Use `clientIp()` from `_clientIp.ts`, and a shared rate-limit store. |
| L-15 | Impersonating names on Aretia pages: Launch Box and the token page never call `isUnsafeName`, there is no list of reserved symbols, and `LOOKALIKE` misses fullwidth and mathematical letters; `cleanText` misses U+061C, U+FEFF and U+2060-2064. | `launchbox/core.ts:53-57`; `walletTokenPublic.ts:166-175, 206`; `tokens/safeText.ts:64` | Yes (`market`) | Normalise with NFKC and a confusables skeleton, reserve ACT, USDC, USDT, SOL, ETH and Aretia for unlisted addresses, and apply the filters on every surface. |
| L-16 | A market-data-only reading can give a green "Established" rating, which wins over admin-power warnings (a score of 50 is still green). | `swings/tokens/risk.ts:119-131`; `market/rowRisk.ts:48-50, 67` | Yes (`market`) | Never show green from market data alone; require a low score and no admin-power warnings. |
| L-17 | The CSP `script-src` is report-only (with `'unsafe-inline'` and no report address), and web3.js is loaded from jsDelivr without SRI. Any injected script defeats every inspector. | `website/vercel.json`; `src/pages/wallet/index.astro:514`; `src/pages/buy.astro:52-53` | — | Self-host or add `integrity`, and enforce a nonce- or hash-based `script-src` and `connect-src` with reporting. (Partly tracked in `readiness.ts:56`.) |

| ID | Title | Location | Reproduced | Recommendation |
|---|---|---|---|---|
| L-18 | Deploy verifiers: the Solana scripts do not check the genesis hash, so a devnet RPC passed with `--cluster mainnet` reports "Verified". The EVM curve verifier does not read back the factory's `feeRecipient()` and `fee()`. | `curve-solana-mainnet.mjs:53-56`; `pool-solana-mainnet.mjs:185-188`; `curve-mainnet.mjs:22-23, 59-66` | No | Check `getGenesisHash` per cluster; read the factory back and check the fee address has no code. |
| L-19 | EVM pool: anyone can give a dust (and locked) position to an address, which then blocks `transferPosition` to it for good. Fully withdrawn positions stay `open`, and "Your pools" truncates to 50 rows before filtering, so spam hides real positions. | `AretiaPool.sol:270-273, 300-319`; `walletYourPoolsEvmFlow.ts:55-87` | Yes (`evm-pool` PoC1) | Allow merging or a two-step transfer, delete empty positions, and filter before truncating. |
| L-20 | EVM pool: no `deadline` on `buy`, `sell`, `add`, `remove` or `createPool`. | `AretiaPool.sol:171, 197, 226, 259` | — | Add a `deadline` and check it in the inspector. |
| L-21 | Solana pool: every swap needs the fee wallet's wSOL associated account. Closing it, which is the normal "unwrap" in Phantom, stops all trading until someone re-creates it. | `aretia-pool lib.rs:535-536`; `walletAretiaPoolTx.ts:190-196` | Yes (`solana-pool`) | Treasury runbook: never close it. Optionally add an idempotent create in swap flows, or use a program-owned protocol vault. |
| L-22 | Solana pool: just-in-time liquidity can take LP fees during the launch-fee window (6.67 of 20 SOL in the PoC). | `aretia-pool lib.rs:214-295`; `pool-math/src/lib.rs:135-143` | Yes (`solana-pool`, figures) | Document it, or add a minimum holding period, or refuse `add_liquidity` while the schedule is active. |
| L-23 | Solana pool: an unlocked owner removing everything leaves `l = 0`, and the pool PDA can never be used or re-created for that mint (this also matters for `aretia-curve-grad`). | `pool-math/src/lib.rs:131, 254-267`; `lib.rs:489` | Yes (`solana-pool`) | Keep a permanent minimum liquidity, or allow re-opening at `l = 0`. |
| L-24 | EVM Send skips Shield's address-poisoning guard and the sent-address book that the Solana Send uses. | `walletApp.ts:801-878` vs `:936-977` | — | Run `guardRecipient` with the same acknowledgement checkbox. |
| L-25 | The Meteora pool inspector checks starting price, liquidity and pool address only against the SDK's own output; a faulty SDK would open the permanent pool at the wrong price. | `walletPoolTx.ts:91-117, 148, 274-279` | — | Compute the Q64.64 price and the pool PDA independently (`meteoraPoolAddress` already exists in `walletCurveTx.ts:47-52`). |
| L-26 | Shield's approval scan leaves out spenders this code itself creates: AllowanceHolder, Permit2's internal allowances, Curve pools and Balancer. | `swings/shield/approvals.ts:43-48` | — | Add them, and read `permit2.allowance`. |

| ID | Title | Location | Reproduced | Recommendation |
|---|---|---|---|---|
| L-27 | Anyone can fill the tracked-pools table, pushing real pools out of the 30 slots per chain. The pools endpoint spends 8 paid `getProgramAccounts` calls per random mint and returns provider error text. | `api/_swingsCandles.ts:91`; `market/tickStore.ts`; `api/_swingsPools.ts:130-157` | — | Track only known pools, set a global budget, check the mint exists first, and drop `keyedReason`. |
| L-28 | The alert service worker accepts protocol-relative URLs (`//evil.example`). | `public/aretia-alerts-sw.js:32` | — | Parse with `new URL(u, origin)` and require the same origin. |
| L-29 | Spam tokens priced from dust pools appear in holdings as large balances that are not marked thin. | `walletApp.ts:224, 252` | — | Carry pool liquidity into the "thin" mark, or set a minimum liquidity before showing a value. |
| L-30 | act-staking: the stored tier keeps its lock multiplier after `unlock_at` and is not refreshed when the tier config changes. Nothing uses tiers yet. | `act-staking lib.rs:91-115, 228-233` | — | Compute the tier on read, and give expired locks no multiplier. |
| L-31 | `aretia-curve-grad` hard-codes the devnet pool program id. The devnet curve (`9HVu…`) and pool (`4EBV…`) programs run pre-fix builds, so they behave differently from the reviewed source. | `aretia-curve-grad/.../lib.rs:51`; `walletCurveCore.ts:16` | — | Gate the id behind a cargo feature, add a check to the deploy script, and redeploy devnet from the current source. |

## 4.4 Informational

### EVM contracts

- **I-1 Documentation does not match the deployment.** `AretiaCurve.sol:5`, `BONDING_CURVE_EVM.md:3`, `contracts-evm/README.md:23` and `walletCurveEvmCore.ts:4, 8-9, 17, 53` say the curve is "NOT deployed anywhere" and the addresses are null, but all three are set. BONDING_CURVE_EVM §12 and §13 say BNB Chain and Polygon are not deployed. `VERIFY.md` has no instructions for verifying the curve.
- **I-2 NatSpec mismatches.** The graduation price check compares against the deposit ratio (`coinIn/tokensIn`), not "the curve's last price" (`:264-266, 347`). `quoteBuy` reverts rather than "returns zeros" (`:303`).
- **I-3 Events.** There is no `SoldOut` event. `Traded` has no `realTok`, and its `coin` is gross in both directions, which is undocumented. `Migrated` omits what was left unused. In the pool, `RecipientSet` emits the raw argument where storage holds 0. There is no `Locked` event at creation, and `Swapped.tokens` means tokens out on a buy but tokens in on a sell.
- **I-4** `MAX_TOTAL` **(2^96) is below what the factory allows**, which forces M-3 for large-supply tokens.
- **I-5 Value stranded for good.** This covers ≤ 0.1% graduation leftovers, an aborted curve's `lpTokens`, k-rounding dust, direct token transfers and forced coin. In the pool: LP-fee rounding dust (`acc` floor; total at extreme `l`) and forced coin.

- **I-6 Gas.** `feeBps`, `createdAt`, `target` and `vSol0` are stored but never read on chain. Checks at `:244, 246, 285` can never fail.
- **I-7** `quoteBuy` **in** `AretiaPool.sol:332-336` **skips the** `MAX` **reserve check that** `_buy` **applies.**
- **I-8 JIT liquidity in the EVM pool** is possible but needs outside inventory; the PoC showed the pool alone is too thin to fund it.
- **I-9 The ERC-20** `approve` **race in AretiaToken.** It is standard, and not reachable through Aretia's flows (they use exact approvals).
- **I-10 Positions owned by addresses that cannot act** (the pool, the token) are orphaned; this affects only the caller.

**Solana programs**

- **I-11** `aretia-curve` **migrate's 0.1% check uses saturating subtraction.** It can be loosened by donating to the migrator's accounts, but the donor always loses money. Requiring that the leftover is at least the starting balance would make the documentation's claim exact.
- **I-12 Stranded value.** This covers migration remainders, donations, the `lp_tokens` of an aborted curve (about 20% of committed supply), rent, and Meteora farming rewards (`claim_reward` is not wrapped).
- **I-13 The solvency floor uses the current** `Rent::get()`**.** A rise in rent would block sells until the vault is topped up. Store the rent paid at creation.
- **I-14 Events are emitted as logs and can be truncated in** `migrate`**.** Consider `emit_cpi!`.
- **I-15 Pool: positions with fewer than 1000 unlocked shares cannot withdraw them** (`pool-math:255`).
- **I-16 Pool: the recipient can be set to the pool PDA**, so a claim self-transfers into the vault (reproduced). Only the claimer loses.
- **I-17 Pool web:** quotes use the browser clock; the Swings adapter marks a topped-out range "inactive" and exposes virtual reserves as if real; the flows close any pre-existing wSOL balance in the user's associated account (refunded to the same wallet).
- **I-18 Documentation drift.** `docs/ARETIA_POOL.md:212` lists transferable positions, wSOL and ranges as "not done", but all three exist. `STAKING_DESIGN.md` says the tier is recomputed on unstake, but unstake closes the account. `act-staking` accepts duplicate durations and has no upper bound on the multiplier.

**Web and server**

- **I-19 Fee details.** Fees round down, so amounts under 173 raw units pay no fee. Fee assets are an exact list, so bridged stablecoins pay no fee. `MANAGEMENT_FEE.md:70` still says 1% on every swap. The legacy Trade tab's `SWAP_FEE_BPS = 100n` is still live (`walletTools.ts:227`). The Solana fee wallet is a single hot key.
- **I-20 The public quote API drops the Permit2 step of Uniswap V4 routes** (`api/_swingsQuote.ts:183`), so integrators' swaps revert.
- **I-21 ed25519 sign-in uses zip215.** A small-order "address" can sign in. This has no user impact.
- **I-22 Secret handling.** Secrets are compared with `!==` rather than in constant time (cron, Telegram, inbox). The inbox secret is passed in the query string. The session

secret falls back to the Supabase service key. Sessions cannot be revoked.

- **I-23 Canary and look-alike claims.** The canary list is unsalted SHA-256 of public addresses, so anyone can check a candidate. The look-alike comment claims "one-in-billions"; it is 1 in 16.7M for EVM hex.
- **I-24 PostgREST filter quoting.** Token search escapes values with backslashes inside `or=(…)` instead of quoting them, so crafted text can change the filter on the public `token_registry`.
- **I-25 Unbounded growth and cache use.** `notify_links`, `swings_records` and `tracked_pools` are never pruned. The OHLCV and trades caches empty themselves when full, so callers can spend the shared GeckoTerminal allowance. The Radar global limit can be used up. `op: test` can spam the Telegram bot.
- **I-26 Client details.**
  - `u64le` wraps silently, although balance checks guard it today (`walletTools.ts:153-157`).
  - The EVM V2 pool fee is requested only after the pool exists.
  - Solana Send re-plans after 45 s.
  - `findFactoryNonce` runs up to 300k hashes on the main thread.
  - `evmCanAbort` uses the local clock.
  - One receipt is treated as final.
  - The Launch Box Verified badge can never appear (`launchBoxVerified.ts:13-15` imports a name that does not exist).
  - Launch Box keeps stale names.
  - The padlock covers liquidity tokens only.
  - The TradingView library is fetched by a mutable tag with no hash.
  - Token symbols reach Telegram as plain text (URL-like symbols become links).
- **I-27 Operator scripts.**
  - `mainnet/gen-deployer.sh:4` overwrites a key with `--force` and no prompt.
  - The `stage-*.sh` scripts leave the global `solana config` pointing at mainnet.
  - `convert-phantom-key.mjs:104` writes the key with the default mode, not 0600.
  - `curve-mainnet.mjs:2` says it "never holds a key", but its `send` mode reads one.
  - `npm audit --omit=dev` reports 3 high and 4 moderate advisories in transitive dependencies (`toml`, `bn.js`, `stream-json`), none reachable from API input.

---

## 5. Previously reported issues: status

From `docs/AUDIT_2026-10-18.md` and git history, re-checked at the reviewed commit:

| # | Issue | Status |
|---|---|---|
| 1 | Solana curve vault ATA squatting blocks `create_curve` | **Fixed** (`init_if_needed`, Anchor ATA constraints). Regression tests in both curve suites. |
| 2 | Solana pool vault squatting and the exact-`y0` check | **Fixed** (handler `create_idempotent` plus raw checks; "rose by y0"). |
| 3 | Graduation pool squatting freezes holders for 7 days | **Mitigated** (`abort_migration` with an optional pool account, which cannot be faked). The squat itself remains possible. |

| # | Issue | Status |
|---|---|---|
| 4 | EVM graduation griefing with dust and `sync()` | **Open, immutable.** It is cheaper than first reported, but a permissionless rescue exists (M-4). |
| 5 | Launch, pool and curve fees credited as swap referrals | **Fixed** (`de0de5a`). |
| 6 | Squatting, creator dumps, sandwiching | **Accepted by design.** M-3 shows the dump also locks buyers out. |
| 7 | Graduated pool fees unclaimable | **Fixed on Solana** (`claim_pool_fees`, SOL only). Unchangeable on deployed EVM curves. |
| 8 | SBF stack overflow in `migrate` and `create_pool` | **Fixed** in all three programs; lockfiles committed. |
| — | 0x AllowanceHolder token and amount decoding; fee only after simulation | **Fixed** (`04eb145`). H-1 and L-2 are what remains. |

---

## 6. Properties checked and found sound

**Curve maths** (`AretiaCurve.sol`, `curve-math`):

- the fee rounds up and outputs round down;
- `vSol − realSol == vSol0` and `vTok − realTok` stay constant;
- k never falls;
- the sell-out buy never charges more than offered (proved: `ceil(n·B/(B−b)) ≤ g`);
- a buy followed by an immediate sell never profits.

A random run of 24,000 steps over six parameter sets, up to total = 2^96, found no violations. No overflow is possible: the bound is about 2^222 against 2^256 on EVM, and k < 2.6e37 against u128 on Solana.

**Pool maths** (`AretiaPool.sol`, `pool-math`):

- the pool is equivalent to a Uniswap v3 range [P_floor, ∞);
- every division rounds against the trader;
- k never falls on a swap, and k per share never falls on add or remove;
- `gross ≤ y`;
- Σ shares = I at all times, with settlement before every share change;
- fee credits never exceed the fees collected;
- no first-depositor or inflation attack is possible.

The web mirror `walletAretiaPoolCore.ts` matches the Rust line for line.

**Solvency:** `balance ≥ totalOwed` (EVM) and `require_solvent` (Solana) hold after every value move. Donations only add.

**Reentrancy (EVM):** every value-moving function holds the lock, and state is written before every external call. Only factory tokens are accepted, verified by CREATE-address derivation (correct for every nonce width).

**Solana account checks:**

- every PDA is checked by seeds and the stored bump;
- `has_one` binds mint, pool and owner;
- token accounts are checked for mint, authority and token program;
- every cross-program call goes to an address-checked program;
- instruction discriminators and Meteora's 107-byte parameter layout match the IDL;
- no type cosplay or seed collision is possible.

**Mint check (Solana programs):** it reads raw bytes: owner Token-2022, no mint or freeze authority, and a TLV walk that accepts only extensions 18 and 19 with an exact length. This refuses every dangerous extension, and extensions cannot be added after initialisation.

`claim_pool_fees`**:**

- anyone may call it, and the destination is fixed;
- only the wSOL change across Meteora's `claim_position_fee` is forwarded;
- principal cannot be touched, because the liquidity is permanently locked.

**Factory:** immutable with no owner; the exact fee is forwarded; supply is minted once; the bytecode matches its artifact byte for byte (rebuilt with solc 0.8.26).

**Web builders:**

- EVM approvals are always the exact amount, to a constant spender, and read back;
- calldata decoders are strict and canonical;
- the launch and curve inspectors re-encode and compare byte for byte;
- Solana transactions are compared byte for byte after the wallet signs, and only one signer is allowed;
- the direct Solana and EVM swap builders build locally, check the owner program of every venue account, and add the fee atomically (Solana);
- Jito tips are allow-listed and bounded;
- two-step EVM routes measure what actually arrived and check the displayed minimum again.

**Server:**

- the RPC proxy has a method allow-list, single-call bodies of at most 16 KB, and forwards no caller headers;
- sessions use HMAC-SHA256 with constant-time comparison;
- every per-user query filters on the session owner (no IDOR);
- RLS is on for every table and only the service role is used, on the server;
- the cron and webhook routes refuse requests when the secret is unset;
- link codes are 128-bit and single-use.

**XSS:** every `innerHTML` writes constant SVG, and all chain and indexer text is set with `textContent`. Links are https-only with `noopener`. `frame-ancestors 'self'` is enforced.

**Secrets:** a search of the full git history found no keys, mnemonics, tokens or API secrets.

---

## 7. Recommendations in priority order

1. **Before anything else (High):**
    - add the AllowanceHolder and Settler checks (H-1) and the Jupiter instruction allow-list (H-2);
    - until then, consider turning aggregators off (`SWINGS_AGGREGATORS=off`) and making L-13 fail closed;
    - stop signing from restored CCTP records (H-3).
2. **Deployed EVM curves:**
    - monitor the fee key's code on all three chains, and never sign a 7702 authorisation with it (M-2);
    - ship the rescue-graduate helper (M-4);
    - show the share of supply held outside the curve (M-3).
3. **Data integrity (live):** apply the raw-mint rule in the Launch Box indexer, the token page and the risk engine (M-6); fix the Verified checker (M-7); rank pools by quote token (M-8).
4. **Server:**
    - normalise referral hashes and dedupe existing rows (M-1);
    - restrict push hosts and read channels in ordered pages (M-11);
    - require a session for ramp URLs (L-4);
    - use standard sign-in formats with server nonces (L-3).
5. **Before mainnet deploys:**
    - `aretia-pool`: fix M-5, L-21 and L-23;
    - `act-staking`: fix M-10;
    - `aretia-curve`: fix L-8 (a one-line change; it needs a rebuild before the pending deploy if it is to be included);
    - `aretia-curve-grad`: fix L-31;
    - deploy scripts: fix L-18.
6. **Defence in depth:** enforce the CSP and add SRI (L-17), bind `chainId` (L-1), and re-confirm a quote that changed when Swap was pressed (M-12).

---

## 8. Appendix: methodology, tooling, reproduction

**Method.**

- Eight reviewers worked in parallel: EVM curve and factory; EVM pool; Solana curves; Solana pool and staking; swap engine; server and database; transaction builders; market, Launch Box and operator tooling.
- Each read its scope line by line against a checklist (access control, arithmetic and rounding, state machines, external calls, token assumptions, MEV, DoS, input validation, data integrity, documentation drift) and wrote proofs of concept where practical.
- Every High and Medium finding was then checked a second time against the source, and its test was re-run.

**Tooling:**

- solc-js 0.8.26 (rebuilt artifacts byte-identical to the committed ones);
- `@ethereumjs/vm` 8 with the real Uniswap V2 factory, Router02 and WETH9 builds from npm;
- LiteSVM with the committed `.so` files;
- Agave `cargo-build-sbf` 2.1.21;
- vitest against the real handlers with fake network and database.

**Reproduction:** see `poc/README.md`. All 15 web proofs of concept pass at commit `5e7fb23`. The EVM scripts print the figures quoted above. The Solana tests are run by copying them into the matching `tests/tests/` folder.

**Not covered:** formal verification; economic simulation beyond the cases given; the behaviour of third-party services under real compromise; live mainnet testing.